

Progression notes

Working log — last updated 2026-09-20. For the project overview, setup, and run commands, see the [README](#). I add to this page as the project grows; the final report will be a trimmed subset of it.

Current progress

Implemented

- **Bilinear reference reader:** A full-resolution sampler that every later representation is read through, so quality differences come from the representation and not the sampling code.
- **S3TC/DXT1 baseline:** Classic 4×4 block compression from scratch. It gives a fixed 6× reduction and lands around 41–43 dB, which is the bar the learned version has to beat.

For easy comparison, both implementations are abstracted to expose the same `compress(texture)` / `sample(u, v)` interface.

To implement next

- A multi-resolution feature grid plus a decoder MLP, fit per texture, then post-training 8-bit quantization.
-

The shared read convention

A texel's center sits at `(i + 0.5) / size`, and a coordinate `u` maps into continuous texel space as `x = u * width - 0.5`, likewise for `v / y`. Lookups outside the texture clamp to the border (replicate padding), matching PyTorch's `F.grid_sample(..., align_corners=False, padding_mode="border")`.

Bilinear sampling

Given (u, v) , find the four surrounding texels, take the fractional position (s, t) inside the cell, and blend:

$$\text{color}(u, v) = (1-s)(1-t) \cdot c_{00} + s(1-t) \cdot c_{10} + (1-s)t \cdot c_{01} + s \cdot t \cdot c_{11}$$

The four corners are named for their offsets: c_{00} is the top-left of the cell and c_{11} the bottom-right. It's vectorized over a batch of coordinates, so the same function handles a few probes or a full megapixel pass.

Correctness validated by:

- Sampling an image at its own texel centers reproduces it exactly (PSNR = ∞), which pins down the texel-center convention.
- On random coordinates it matches `F.grid_sample` (`align_corners=False`, `padding_mode="border"`) to floating-point tolerance, borders included.

S3TC / DXT1 block compression

S3TC splits the image into 4x4 blocks and spends a fixed 64 bits per block:

- Two endpoint colors, each a 16-bit RGB565 value ($2 \times 16 = 32$ bits).
- Sixteen 2-bit indices, one per texel ($16 \times 2 = 32$ bits).

In total, it takes 8 bytes per 16 texels, or 4 bits per texel. This is **6x smaller** than 24-bit RGB. A four-color palette is interpolated from the endpoints, and each texel stores the index of its nearest palette color:

$$c_2 = (2c_0 + c_1) / 3 \quad c_3 = (c_0 + 2c_1) / 3$$

When decoding, the `sample(u, v)` path decodes the image once and then bilinearly samples it with the reader above, so S3TC is read exactly like every other representation.

Picking the endpoints

The endpoints dominate block quality, so I choose them per block in two steps:

1. **Principal color axis (PCA).** Subtract the block mean, approximate the dominant direction of variation with a few power iterations on the 3×3 covariance, project the 16 texels onto that axis, and take the two extremes. This lines the palette up with the direction the block actually varies in.
2. **Least-squares refit.** Assign each texel to its nearest palette entry, then solve for the endpoints that best reconstruct those colors. Palette entries are fixed blends of the endpoints (`c2` and `c3` above), so each block is a tiny 2-variable least-squares problem with a closed form. Repeat this assign-and-refit twice.

Flat blocks, where the fit is underdetermined, just keep their previous endpoints instead of dividing by a near-zero determinant. Final endpoints are quantized to RGB565, and blocks are always written in the 4-color mode with the larger code first.

RGB565, and why the divide-by-levels shortcut isn't exact

Each endpoint channel is quantized by multiplying by its level count, rounding, and dividing back:

```
levels = (31, 63, 31)           # 5, 6, 5 bits for R, G, B
q      = round(c * levels)       # integer 5-6-5 code
c_hat  = q / levels              # back to [0, 1]
```

This is close to, but not exactly, how hardware maps a 5- or 6-bit code back to a value. Hardware expands a code by *bit replication* into 8 bits, so the true value would be `round(q * 255 / 31) / 255`. The shortcut implemented instead returns `q / 31`. Since $255/31$ isn't an integer, bit replication lands on one of 256 fixed 8-bit levels, while `q / levels` can fall between them. The difference is under a $1/255$

step and both maps are monotonic with the same half-step rounding error, so the difference is not significant.

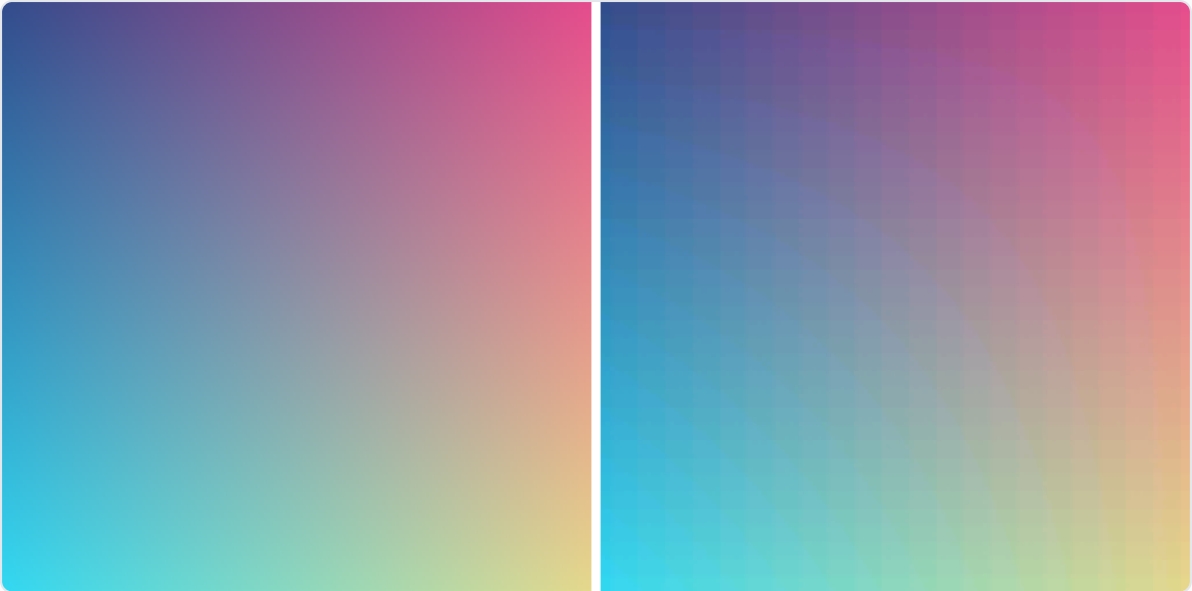
Why does green get the extra bit? The 5-6-5 split puts six bits on green, one more than red or blue. This is because green carries most of the human eye's perceived luminance (which is probably because sunlight is mostly green which is because green is between blue and red in terms of light wavelength). The Rec. 709 weights are about 0.21 R, 0.72 G, 0.07 B, so a green error is 3x as visible as the same error in red and 10x as visible as one in blue. Spending the extra level on green therefore removes the most visible banding per bit. From what I've seen, every common 16-bit RGB layout favors green because of this.

Results

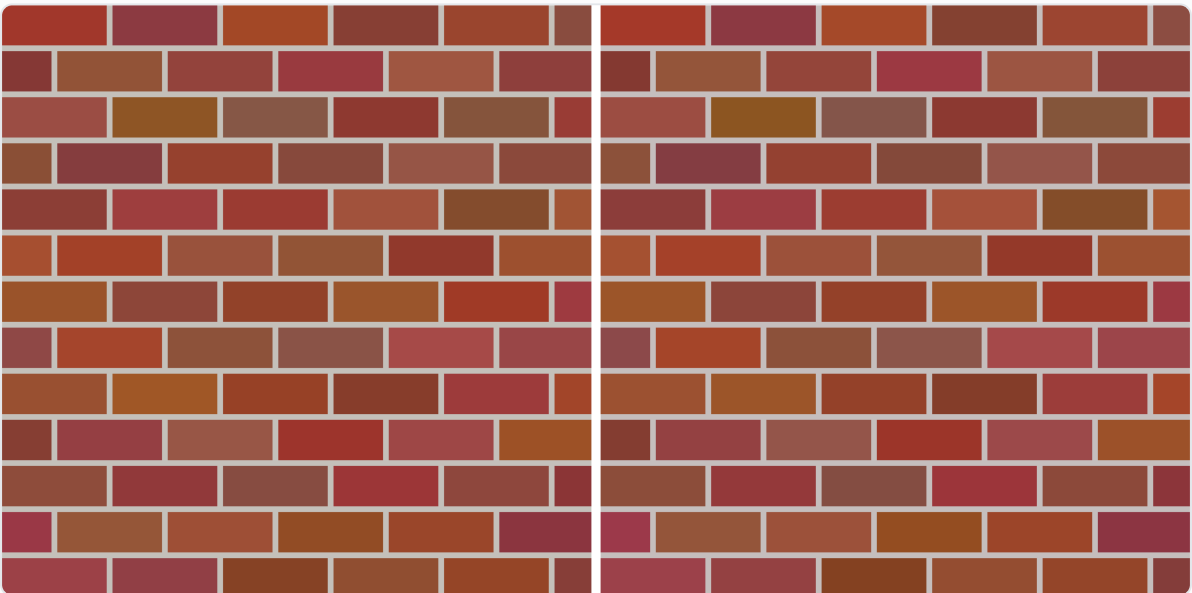
S3TC is fixed-ratio, so all three textures compress to exactly the same size. PSNR is measured on the decoded image against the original, both normalized to [0, 1] , with $PSNR = -10 \cdot \log_{10}(MSE)$.

Texture	PSNR (dB)	S3TC size	Original size	Compression
gradient (smooth, low-frequency)	42.89	128 KB	768 KB	6.00× smaller
bricks (structured, sharp edges)	41.39	128 KB	768 KB	6.00× smaller
clouds (fractal, multi-scale)	42.63	128 KB	768 KB	6.00× smaller

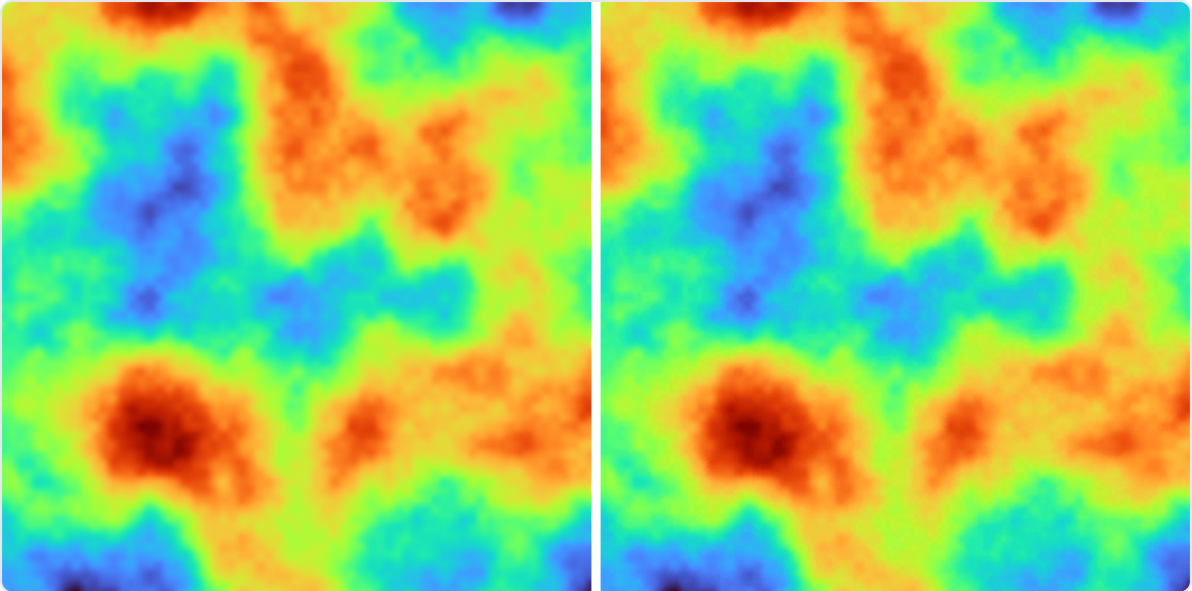
"Compression" is raw size divided by compressed size (a 6× reduction); the compressed file is one sixth of the original.



Gradient: original (left) vs. S3TC (right), 42.89 dB. The smooth ramp is the hardest case for a block format — the reconstruction shows faint block stepping.



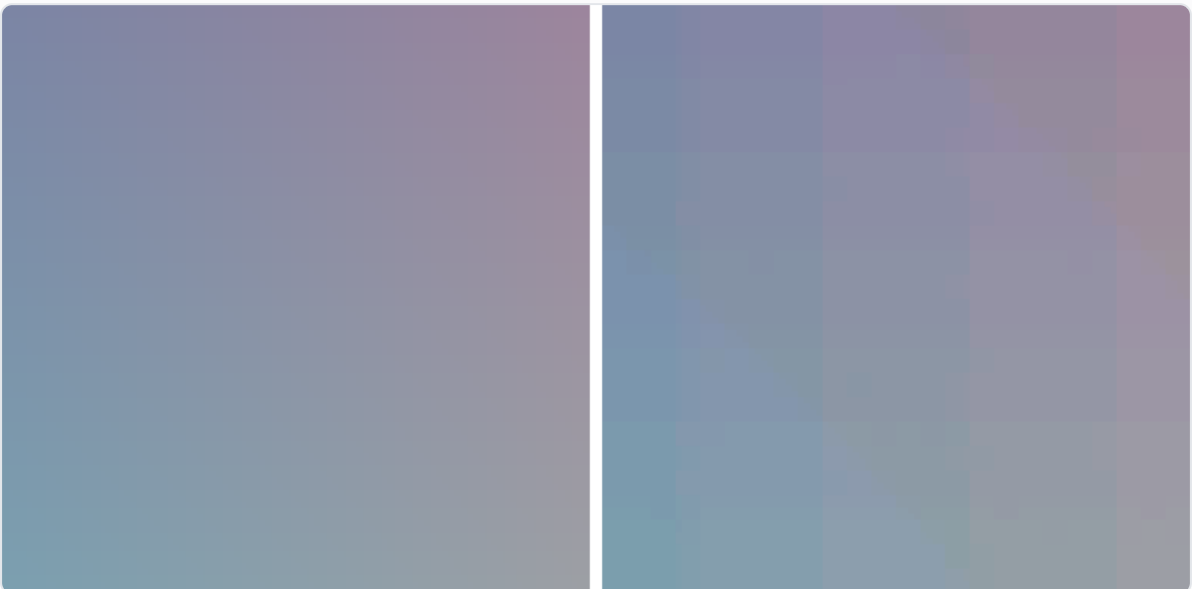
Bricks: original (left) vs. S3TC (right), 41.39 dB. Brick faces and mortar survive, but their colors shift slightly inside each block.



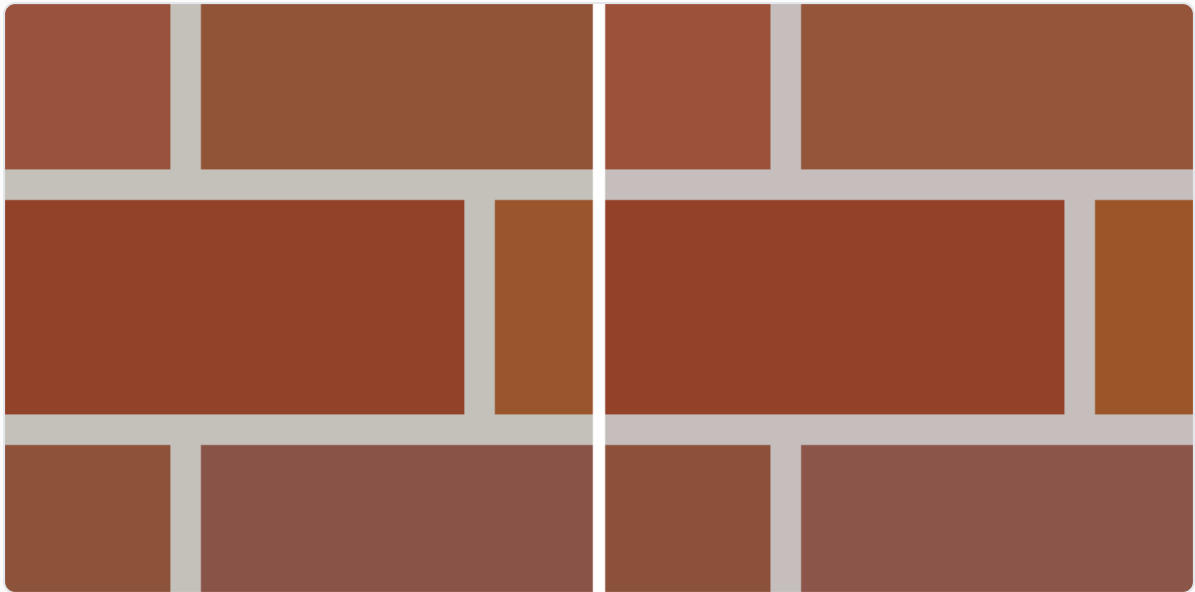
Clouds: original (left) vs. S3TC (right), 42.63 dB. The soft, low-contrast detail mostly survives, with subtle blocking.

S3TC is sometimes not good

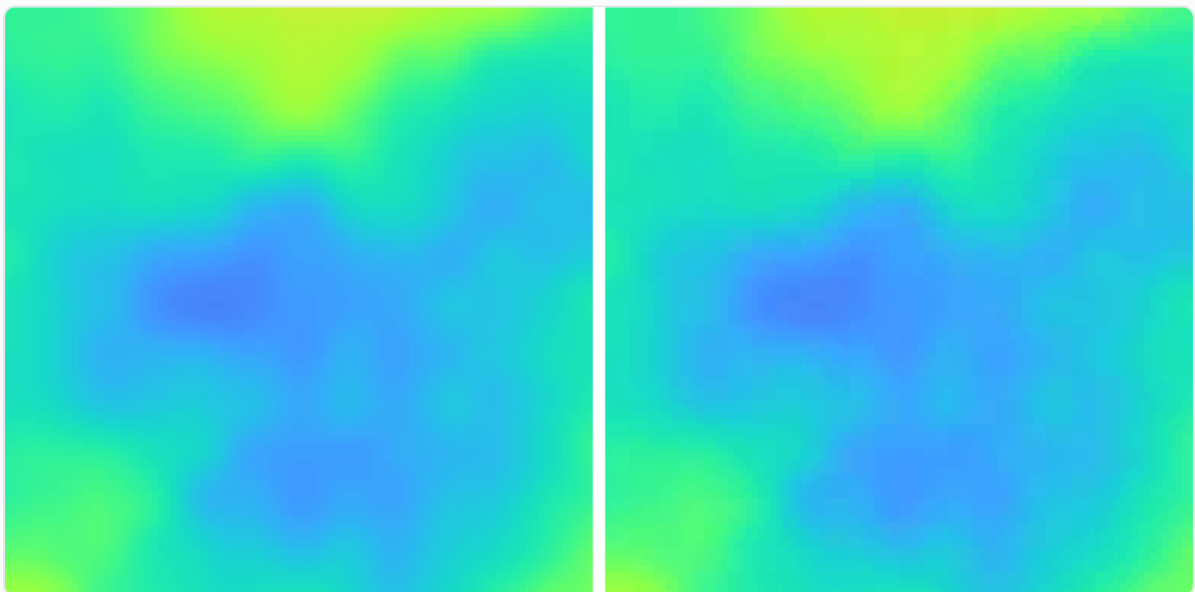
The clearest failure is smooth gradients. The crops below are 96×96 texels, nearest-neighbor upscaled 4×, so each 4×4 block becomes a 16×16 square. Figures are regenerated by `scripts/make_figures.py`.



Gradient: the original ramps smoothly; S3TC shows block-to-block banding.



Bricks: flat brick color picks up blocky shading from the palette fit.



Clouds: soft detail survives; faint blocking in low-contrast areas.

This behavior is because of the nature of S3TC and cannot really be improved.

- The palette is four points on the segment between two endpoints. A smooth image varies in two dimensions inside a block, but the block can only represent colors near that 1-D line, so the leftover error shows up as a color step at every block boundary, causing the banding.

- High-frequency detail that doesn't align with the 4×4 grid can't be represented; edges get smeared or stepped.
- Every block costs the same 64 bits whether it's flat or busy, and nothing is shared between blocks.
- RGB565 rounds each endpoint to 5 or 6 bits before the palette is even built, adding a second error source on top of the palette fit.

At this stage the baseline is 128 KB (6×) at roughly 41–43 dB. A learned representation will replace the fixed palette with a shared feature grid plus an MLP, so its size will be tunable instead of fixed. Because of this, I will compare them by matching size then comparing PSNR and matching PSNR then comparing size.

Log

2026-09-20 Project scaffold, environment, and CUDA setup. Bilinear sampler with the shared read convention, verified against `grid_sample`. S3TC/DXT1 baseline with PCA endpoints and least-squares refits; 6× fixed at ~41–43 dB. Figures and this page added.
